•¿Qué es GitHub?

Pagina que me permite subir mi código, recurrir a cambios y acceder a los mismos. También me permite acceder al código de los demás y poder usarlo

•¿Cómo crear un repositorio en GitHub?

Registrandome en la pagina de GitHub y haciendo click en Your repositories” y luego "New"

•¿Cómo crear una rama en Git?

Con el código git Branch y un nombre de la Branch que el usuario desee

•¿Cómo cambiar a una rama en Git?

Con el código Git checkout y el nombre de la rama en la cual se quiera posicionar

•¿Cómo fusionar ramas en Git?

Son como líneas temporales en nuestro código a las cual podemos acceder

•¿Cómo crear un commit en Git?

Con el código git commit -m “aca escribir opcionalmente algo que me ayude a identificar el commit”

• ¿Cómo enviar un commit a GitHub?

Primero con el código “git remote add origin” < y agregando la URL del repositorio>

Y luego con el código git push origin

• ¿Qué es un repositorio remoto?

Un lugar donde se guarda una versión de mi código alojada en un servidor

• ¿Cómo agregar un repositorio remoto a Git?

con el código “git remote add origin” < y agregando la URL del repositorio>

• ¿Cómo empujar cambios a un repositorio remoto?

Con el código git push origin

• ¿Cómo tirar de cambios de un repositorio remoto?

Con el código git pull

• ¿Qué es un fork de repositorio?

Es una copia a mi cuenta de Github de un repositorio creado por otra persona para que la pueda utilizar yo

• ¿Cómo crear un fork de un repositorio?

Yendo al repositorio deseado luego hacer click en el botón “Fork” y seleccionar mi cuenta

• ¿Qué es un etiqueta en Git?

Funciona como un comentario en la historia de un repositorio son estáticas no cambian o evolucionan como las ramas

• ¿Cómo crear una etiqueta en Git?

Con el código git tag -a <nombre_etiqueta> -m "Mensaje"

• ¿Cómo enviar una etiqueta a GitHub?

Con el código git tag y el <nombre_etiqueta>

• ¿Qué es un historial de Git?

Son los cambios cronológicos hechos en mi código que figuran en mi repositorio

• ¿Cómo ver el historial de Git?

Con el comando git log

• ¿Cómo buscar en el historial de Git?

Con el comando git log <nombre_archivo>

• ¿Cómo borrar el historial de Git?

Con el código git revert

• ¿Qué es un repositorio privado en GitHub?

Un repositorio al cual solo yo puedo acceder y ver

• ¿Cómo crear un repositorio privado en GitHub?

Eligiendo la opción de privado a la hora de crear un repositorio

• ¿Cómo invitar a alguien a un repositorio privado en GitHub?

Una vez en mi repositorio en la pestana de configuración puedo agregar colaboradores

• ¿Qué es un repositorio público en GitHub?

Un repositorio el cual otras personas pueden ver y acceder

• ¿Cómo crear un repositorio público en GitHub?

Eligiendo la opción de público a la hora de crear un repositorio

• ¿Cómo compartir un repositorio público en GitHub?

Compartiendo la URL del repositorio público de Git Hub

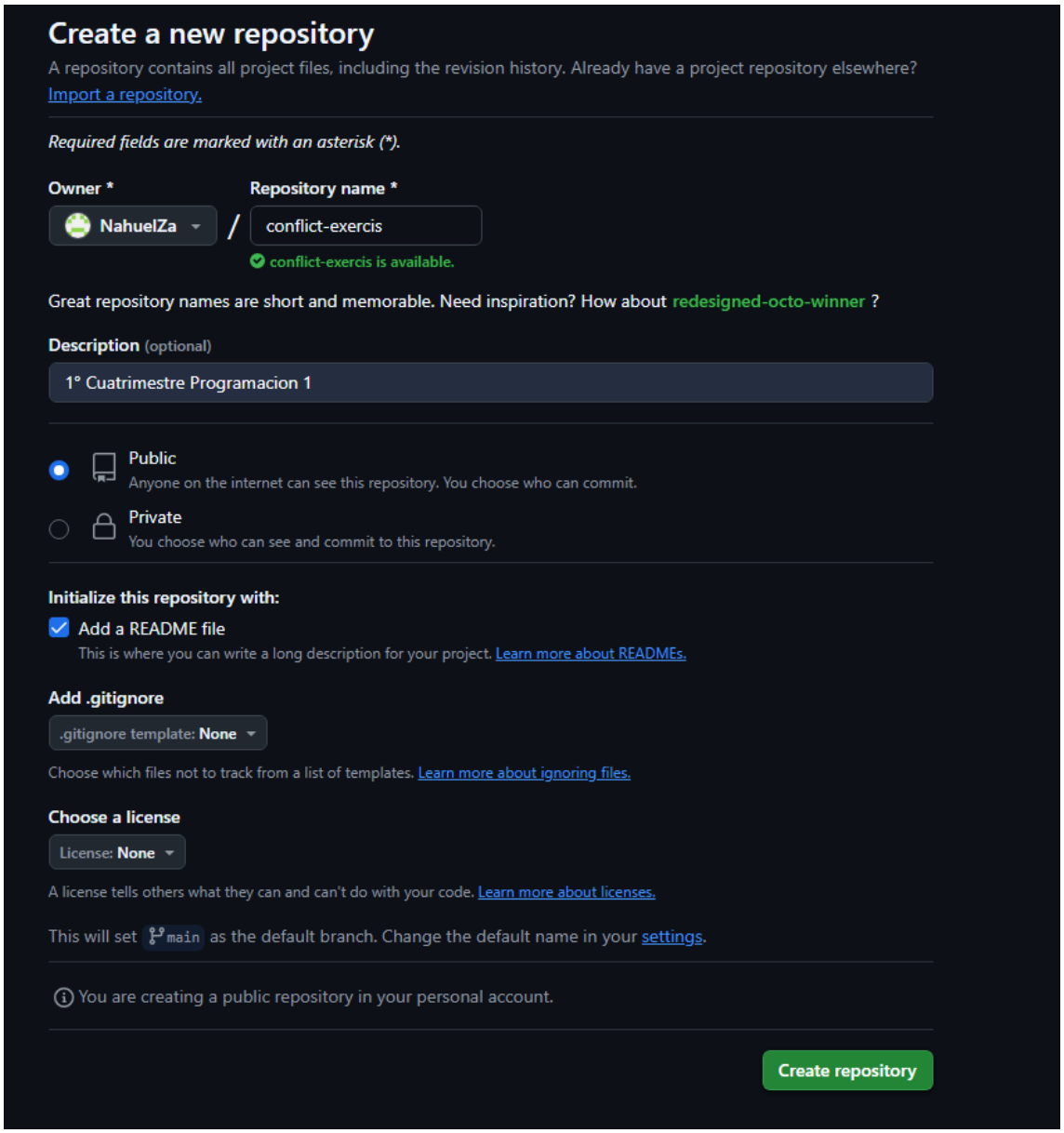
2) Realizar la siguiente actividad:

• Crear un repositorio.

o Dale un nombre al repositorio.

o Elije el repositorio sea público.

o Inicializa el repositorio con un archivo.



- Agregando un Archivo o Crea un archivo simple, por ejemplo, "mi-archivo.txt".

o Realiza los comandos git add . y git commit -m "Agregando mi-archivo.txt" en la línea de comandos.

```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git add .
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git commit -m "Agregando mi-archivo.txt"
[detached HEAD f6345ec] Agregando mi-archivo.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename 02 Trabajo Colaborativo/{README.md => mi-archivo.txt} (100%)
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo>
```

o Sube los cambios al repositorio en GitHub con git push origin main (o el nombre de la rama correspondiente).

```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 350 bytes | 350.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/NahuelZa/UTN-TUPaD-P1.git
   6287529..20756ef  main -> main
```

• Creando Branchs

o Crear una Branch

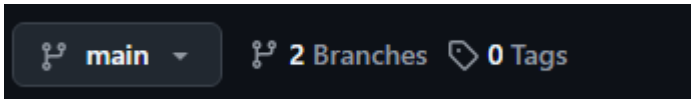
```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git branch new
```

o Realizar cambios o agregar un archivo

```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git commit -m "cambios"
[main aa3ec93] cambios
 1 file changed, 1 insertion(+), 1 deletion(-)
```

o Subir la Branch

```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\UTN-TUPaD-P1\02 Trabajo Colaborativo> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 343 bytes | 343.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/NahuelZa/UTN-TUPaD-P1.git
   20756ef..aa3ec93  main -> main
```



3) Realizar la siguiente actividad:

Paso 1: Crear un repositorio en GitHub

• Ve a GitHub e inicia sesión en tu cuenta.

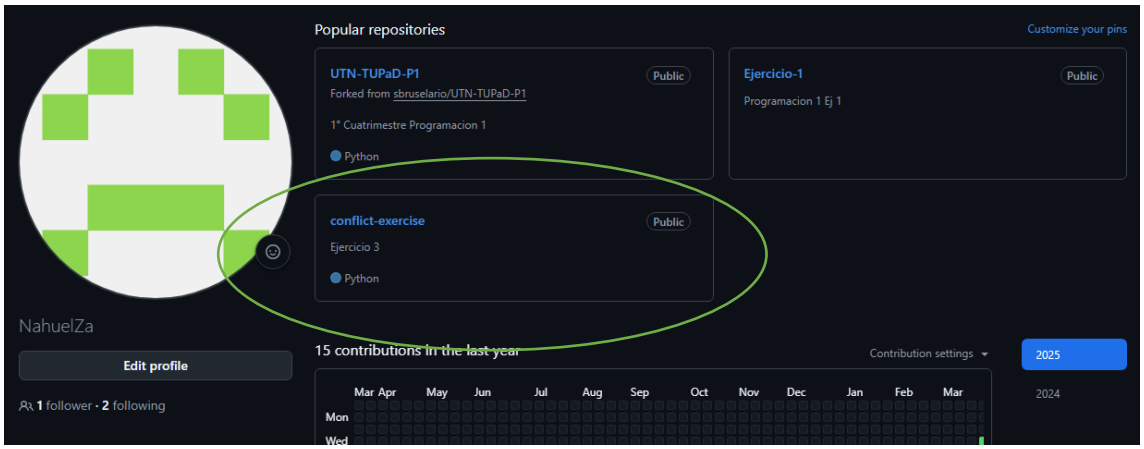
• Haz clic en el botón "New" o "Create repository" para crear un nuevo repositorio.

• Asigna un nombre al repositorio, por ejemplo, conflict-exercise.

• Opcionalmente, añade una descripción.

• Marca la opción "Initialize this repository with a README".

• Haz clic en "Create repository".



Paso 2: Clonar el repositorio a tu máquina local

• Copia la URL del repositorio (usualmente algo como https://github.com/tuusuario/conflict-exercise.git).

• Abre la terminal o línea de comandos en tu máquina.

• Clona el repositorio usando el comando: git clone https://github.com/tuusuario/conflict-exercise.git

• Entra en el directorio del repositorio: cd conflict-exercise

```
PS C:\Users\nahue\Desktop\UTN\Programacion 1\EJercicios> git clone https://github.com/NahuelZa/conflict-exercise.git
Cloning into 'conflict-exercise'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 28 (delta 3), reused 20 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), 176.78 KiB | 9.82 MiB/s, done.
Resolving deltas: 100% (3/3), done.
PS C:\Users\nahue\Desktop\UTN\Programacion 1\EJercicios>
```

Paso 3: Crear una nueva rama y editar un archivo

```
PS C:\Users\nahue\Desktop\Python> git branch feature-branch
PS C:\Users\nahue\Desktop\Python> git checkout feature-branch
Switched to branch 'feature-branch'
PS C:\Users\nahue\Desktop\Python> git branch
* feature-branch
  master
```

• Crea una nueva rama llamada feature-branch: git checkout -b feature-branch

• Abre el archivo README.md en un editor de texto y añade una línea nueva, por ejemplo: Este es un cambio en la feature branch.

• Guarda los cambios y haz un commit: git add README.md git commit -m "Added a line in feature-branch"

```
1  Este es un cambio en la feature branch
```

```
PS C:\Users\nahue\Desktop\Python> git commit -m " cambios en la feature branch"
[feature-branch badd7e4]  cambios en la feature branch
 5 files changed, 59 insertions(+)
 create mode 100644 Python Cheat Sheet 1.pdf
 create mode 100644 Python Cheat Sheet 2.pdf
 create mode 100644 Semana_1.py
 create mode 160000 conflict-exercise
 create mode 100644 hello.py
```

Paso 4: Volver a la rama principal y editar el mismo archivo

• Cambia de vuelta a la rama principal (main): git checkout main

• Edita el archivo README.md de nuevo, añadiendo una línea diferente: Este es un cambio en la main branch.

• Guarda los cambios y haz un commit: git add README.md git commit -m "Added a line in main branch"

```
1  Este es un cambio en la main branch
```

```
PS C:\Users\nahue\Desktop\Python> git branch
  feature-branch
* master
```

```
PS C:\Users\nahue\Desktop\Python> git commit -m "cambios en la main"
[master 66e5474] cambios en la main
 1 file changed, 1 insertion(+)
 create mode 160000 conflict-exercise
```

Paso 5: Hacer un merge y generar un conflicto

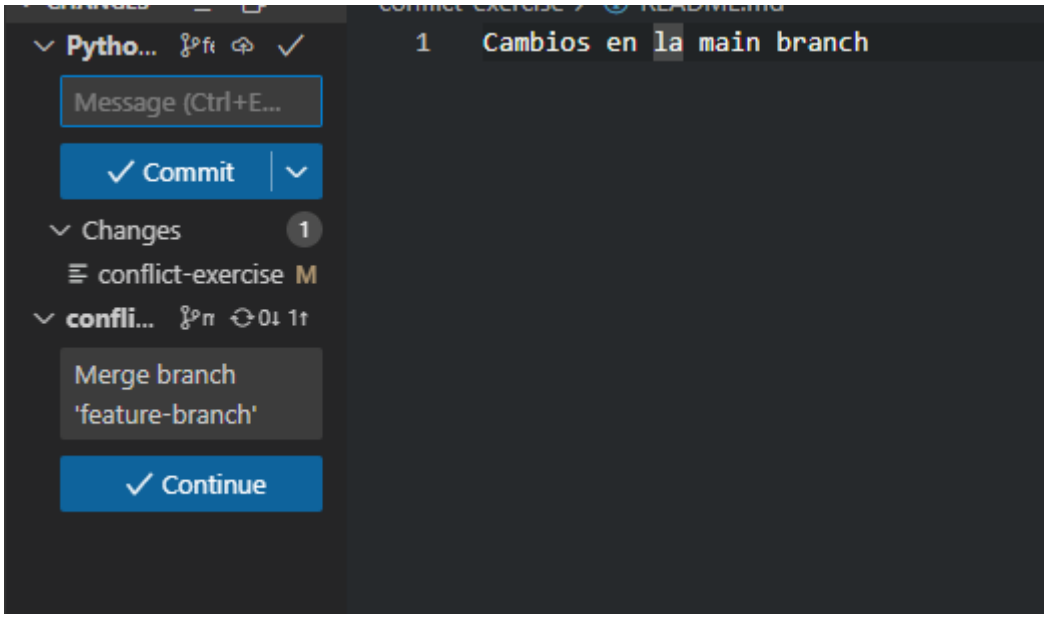
• Intenta hacer un merge de la feature-branch en la rama main: git merge feature-branch

• Se generará un conflicto porque ambos cambios afectan la misma línea del archivo README.md.

```
PS C:\Users\nahue\Desktop\Python> git merge feature-branch
Merge made by the 'ort' strategy.
```

Paso 6: Resolver el conflicto

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
1  <<<<<<< HEAD (Current Change)
2  Cambios en la main branch
3  =======
4  Este es un cambio en la feature branch
5  >>>>>>> feature-branch (Incoming Change)
6
```



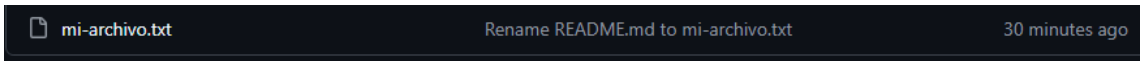
• Abre el archivo README.md en tu editor de texto. Verás algo similar a esto: <<<<<<< HEAD Este es un cambio en la main branch. ======= Este es un cambio en la feature branch. >>>>>>> feature-branch

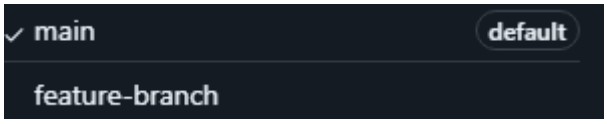
• Decide cómo resolver el conflicto. Puedes mantener ambos cambios, elegir uno de ellos, o fusionar los contenidos de alguna manera.

• Edita el archivo para resolver el conflicto y guarda los cambios(Se debe borrar lo marcado en verde en el archivo donde estes solucionando el conflicto. Y se debe borrar la parte del texto que no se quiera dejar).

• Añade el archivo resuelto y completa el merge: git add README.md git commit -m "Resolved merge conflict"

Paso 7: Subir los cambios a GitHub





• Sube los cambios de la rama main al repositorio remoto en GitHub: git push origin main

• También sube la feature-branch si deseas: git push origin feature-branch

Paso 8: Verificar en GitHub

• Ve a tu repositorio en GitHub y revisa el archivo README.md para confirmar que los cambios se han subido correctamente.

• Puedes revisar el historial de commits para ver el conflicto y su resolución.